

WorkFlow — Requisitos do Sistema

1. Visão do Produto

O **WorkFlow** é uma plataforma web multiempresa para gerenciamento de projetos, tarefas, comunicação interna e acompanhamento de atividades de equipes.

O sistema foi pensado para permanecer disponível durante toda a jornada de trabalho, servindo como uma central de acompanhamento das atividades da empresa.

Ao iniciar o trabalho, o usuário deverá conseguir identificar rapidamente:

- projetos dos quais participa;
- tarefas atribuídas a ele;
- tarefas disponíveis para serem assumidas;
- prazos;
- tarefas atrasadas;
- atualizações recentes;
- comentários;
- mensagens;
- notificações.

O sistema deverá ser genérico e não depender de regras específicas de uma única empresa.

2. Objetivos

O WorkFlow deverá:

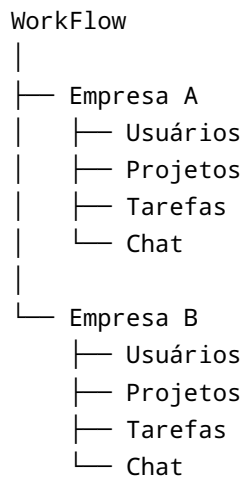
- centralizar o acompanhamento dos projetos da empresa;
 - permitir o gerenciamento das tarefas das equipes;
 - permitir acompanhamento do progresso dos projetos;
 - facilitar a comunicação interna;
 - manter histórico das alterações importantes;
 - permitir rastreabilidade das ações realizadas;
 - fornecer notificações sobre eventos relevantes;
 - permitir a utilização por múltiplas empresas com isolamento de dados;
 - possuir arquitetura segura, testável e escalável.
-

3. Multiempresa

O WorkFlow será uma aplicação **multi-tenant**.

Cada empresa cadastrada no sistema será considerada um **Tenant**.

Exemplo:



Os dados de uma empresa não poderão ser acessados por usuários de outra empresa.

4. Perfis de Usuário

4.1 SYSTEM_ADMIN

Administrador da plataforma WorkFlow.

Responsável pela administração global da aplicação.

Poderá:

- administrar empresas;
- ativar ou desativar empresas;
- realizar operações administrativas globais;
- acessar funcionalidades necessárias à manutenção da plataforma.

O SYSTEM_ADMIN não representa um funcionário comum de uma empresa cliente.

4.2 TENANT_ADMIN

Administrador de uma empresa específica.

Poderá:

- administrar usuários da empresa;
- cadastrar ou convidar usuários;
- ativar ou desativar usuários;
- criar projetos;

- administrar projetos;
- administrar gerentes;
- administrar membros;
- administrar permissões dentro da própria empresa.

Não poderá acessar dados pertencentes a outro Tenant.

4.3 PROJECT_MANAGER

Usuário responsável pelo gerenciamento de projetos.

Poderá:

- criar projetos;
 - administrar projetos sob sua responsabilidade;
 - adicionar ou remover membros;
 - definir responsável pelo projeto;
 - criar tarefas;
 - atribuir tarefas;
 - definir prioridades;
 - definir prazos;
 - validar tarefas;
 - conceder determinadas permissões dentro dos projetos que administra.
-

4.4 MEMBER

Usuário comum da equipe.

Poderá:

- visualizar projetos dos quais participa;
- visualizar tarefas relacionadas a seus projetos;
- executar tarefas atribuídas;
- alterar estados permitidos de suas tarefas;
- adicionar comentários;
- participar do chat da empresa;
- receber notificações.

Um membro poderá receber permissões adicionais em projetos específicos.

5. Permissões por Projeto

Além do perfil global do usuário, o sistema deverá permitir permissões específicas por projeto.

Exemplos de permissões:

- visualizar projeto;
- editar projeto;
- administrar membros;
- criar tarefas;
- assumir tarefas;
- editar determinadas informações;
- validar tarefas.

Um MEMBER poderá possuir mais privilégios em determinado projeto sem precisar se tornar PROJECT_MANAGER globalmente.

6. Projetos

6.1 Criação

Poderão criar projetos:

- TENANT_ADMIN;
- PROJECT_MANAGER.

O criador deverá automaticamente se tornar membro do projeto.

Um PROJECT_MANAGER que cria um projeto poderá assumir sua administração.

6.2 Edição

Poderão editar um projeto:

- TENANT_ADMIN;
 - PROJECT_MANAGER autorizado;
 - responsável pelo projeto;
 - MEMBER com permissão específica.
-

6.3 Responsável pelo Projeto

Um projeto poderá possuir um responsável principal.

O responsável deverá obrigatoriamente ser membro do projeto.

O responsável poderá ser:

- TENANT_ADMIN;
- PROJECT_MANAGER;

- MEMBER autorizado.

Inicialmente, cada projeto possuirá no máximo um responsável principal.

6.4 Membros

Um projeto poderá possuir múltiplos membros.

Não haverá limite fixo de membros no MVP.

Os membros deverão ser adicionados por usuários autorizados.

Um usuário não poderá adicionar a si próprio em um projeto sem autorização, exceto nos casos permitidos pelas regras administrativas.

6.5 Status do Projeto

Os estados iniciais serão:

PLANNING
IN_PROGRESS
PAUSED
COMPLETED
ARCHIVED

Transições deverão respeitar regras de negócio.

Um projeto concluído poderá retornar para `IN_PROGRESS`.

Um projeto arquivado será tratado como fora da operação cotidiana.

6.6 Arquivamento

Projetos não serão excluídos normalmente.

A operação padrão será:

ARCHIVE

Projetos arquivados:

- não aparecerão nas listagens normais de projetos ativos;
- permanecerão disponíveis para consultas administrativas;

- manterão seus dados;
- manterão suas tarefas;
- manterão seu histórico;
- poderão ser utilizados em relatórios posteriores.

Exclusão física será considerada apenas para operações administrativas excepcionais.

7. Tarefas

Toda tarefa deverá pertencer obrigatoriamente a um projeto.

Uma tarefa poderá conter inicialmente:

- título;
 - descrição;
 - status;
 - prioridade;
 - prazo;
 - responsável;
 - colaboradores;
 - data de criação;
 - data de atualização.
-

8. Criação de Tarefas

Poderão criar tarefas:

- TENANT_ADMIN;
 - PROJECT_MANAGER responsável pelo projeto;
 - responsável/líder do projeto;
 - MEMBER com permissão específica para criação de tarefas.
-

9. Responsabilidade da Tarefa

Uma tarefa poderá possuir:

- nenhum responsável;
- um responsável principal;
- zero ou mais colaboradores.

Uma tarefa não poderá possuir mais de um responsável principal.

O responsável representa quem responde diretamente pela entrega.

Os colaboradores representam usuários que auxiliam na execução.

10. Tarefas Disponíveis

Uma tarefa poderá ser criada sem responsável.

Usuários com permissão específica poderão assumir uma tarefa disponível.

Exemplo:

```
Tarefa sem responsável
      ↓
Usuário autorizado seleciona
"Assumir tarefa"
      ↓
Usuário passa a ser responsável
```

11. Edição de Tarefas

Informações administrativas como:

- título;
- descrição;
- prioridade;
- prazo;
- responsável;
- colaboradores;

deverão ser alteradas apenas por usuários com permissão adequada.

O responsável comum pela tarefa não poderá alterar livremente essas informações.

12. Status das Tarefas

Os estados iniciais serão:

```
BACKLOG
TODO
IN_PROGRESS
PAUSED
VALIDATION
DONE
```

Fluxo principal:

BACKLOG
↓
TODO
↓
IN_PROGRESS
↓
VALIDATION
↓
DONE

PAUSED será um estado temporário utilizado quando a tarefa estiver impedida de continuar.

13. Pausa de Tarefa

Ao mover uma tarefa para **PAUSED**, deverá ser informado um motivo.

Exemplo:

Status:
PAUSED

Motivo:
"Aguardando resposta do cliente."

A informação deverá ser registrada no histórico.

14. Validação de Tarefas

Um MEMBER responsável não poderá concluir diretamente uma tarefa.

O fluxo será:

IN_PROGRESS
↓
VALIDATION

Um usuário autorizado deverá então validar a tarefa.

Se aprovada:

VALIDATION



DONE

Se rejeitada:

VALIDATION



IN_PROGRESS

Uma rejeição deverá possuir justificativa obrigatória.

Poderão validar tarefas usuários autorizados, como:

- TENANT_ADMIN;
- PROJECT_MANAGER;
- responsável/líder do projeto;
- usuário com permissão específica.

15. Arquivamento de Tarefas

Tarefas não serão excluídas normalmente.

A operação padrão será o arquivamento.

Uma tarefa arquivada deverá permanecer disponível para:

- histórico;
- auditoria;
- relatórios;
- consultas administrativas.

16. Subtarefas

Subtarefas não farão parte do MVP inicial.

Serão consideradas uma evolução futura.

Exemplo futuro:

Implementar autenticação



└─ Criar entidade User

- └─ Criar endpoint de login
- └─ Implementar token
- └─ Criar testes

17. Dependências entre Tarefas

Dependências formais entre tarefas não farão parte do MVP.

Inicialmente o estado `PAUSED` e seu motivo poderão ser utilizados para indicar impedimentos.

Futuramente será possível implementar relações estruturadas como:

```
Tarefa B
  ↓
depende de
  ↓
Tarefa A
```

Isso permitirá identificar automaticamente tarefas bloqueadas.

18. Comentários de Tarefas

Usuários autorizados poderão adicionar comentários às tarefas.

O autor poderá editar ou remover seu próprio comentário apenas durante uma janela limitada após sua criação.

Após esse período, o comentário deverá permanecer imutável para o autor comum.

Operações administrativas deverão manter rastreabilidade.

19. Histórico e Auditoria

O sistema deverá registrar alterações importantes.

Exemplos:

- criação de projeto;
- mudança de status;
- alteração de prazo;
- alteração de prioridade;
- atribuição de responsável;

- alteração de membros;
- pausa;
- retorno de pausa;
- envio para validação;
- aprovação;
- rejeição;
- arquivamento.

O histórico deverá permitir identificar:

- quem realizou a ação;
- quando;
- qual recurso foi alterado;
- valor anterior, quando aplicável;
- novo valor, quando aplicável.

20. Feed de Atividades

O sistema deverá apresentar atividades recentes relacionadas aos projetos relevantes para o usuário.

Exemplos:

João concluiu "Criar banco".

Maria iniciou "Tela de login".

Lucas comentou em "API de clientes".

O feed será diferente do histórico técnico completo.

Seu objetivo será fornecer uma visão rápida das atividades da equipe.

21. Chat da Empresa

Cada Tenant terá inicialmente um chat geral.

Todos os usuários ativos da empresa poderão utilizá-lo.

O chat será utilizado para comunicação rápida e temporária.

Não deverá ser considerado substituto para:

- documentação;
- comentários permanentes;
- decisões oficiais;

- histórico de tarefas.

22. Retenção das Mensagens do Chat

Uma mensagem permanecerá disponível durante 7 dias contados a partir de sua criação.

Depois desse período, deixará de ser disponibilizada aos usuários.

A implementação poderá utilizar:

CreatedAt
ExpiresAt

A remoção física poderá ser realizada posteriormente por processamento em background.

23. Edição e Exclusão de Mensagens do Chat

RN-CHAT-03

O autor poderá editar ou remover sua própria mensagem por até **5 minutos após o envio**.

Após esse período, o usuário comum não poderá modificá-la.

Usuários administrativos poderão realizar operações de moderação quando necessário, mantendo registro da ação.

24. Menções no Chat

O sistema deverá permitir menções a usuários.

Exemplo:

@Lucas consegue revisar a tarefa?

Uma menção deverá gerar uma notificação para o usuário mencionado.

25. Chat — Funcionalidades Futuras

Não farão parte do MVP inicial:

- anexos;
- chat privado;
- grupos privados;
- chat por projeto.

26. Notificações

O WorkFlow deverá possuir sistema persistente de notificações.

As notificações deverão existir independentemente da conexão em tempo real.

Fluxo:

```
Evento
↓
Notificação salva
↓
Tentativa de entrega em tempo real
```

27. Usuário Online

Quando o usuário estiver conectado:

```
Evento
↓
Notificação persistida
↓
SignalR
↓
Notificação apresentada imediatamente
```

28. Usuário Offline

Quando o usuário estiver offline:

```
Evento
↓
Notificação persistida
↓
Usuário abre WorkFlow posteriormente
↓
Aplicação consulta notificações
↓
Notificação apresentada
```

Dessa forma, nenhuma notificação importante dependerá exclusivamente do SignalR.

29. Tipos Iniciais de Notificação

O sistema deverá considerar inicialmente:

```
TASK_ASSIGNED
TASK_WAITING_VALIDATION
TASK_VALIDATION_REJECTED
TASK_COMMENTED
USER_MENTIONED
TASK_DUE_SOON
TASK_OVERDUE
```

30. Relevância de Notificações

Notificações não deverão ser enviadas indiscriminadamente para todos os usuários.

Somente usuários relacionados ao evento deverão recebê-las.

Exemplos:

Tarefa atribuída:

```
Responsável → recebe
```

Tarefa aguardando validação:

```
Usuários autorizados a validar → recebem
```

Menção:

Usuário mencionado → recebe

31. Estado das Notificações

Uma notificação deverá permitir identificar:

- se foi lida;
- quando foi lida;
- se foi removida da lista pessoal;
- quando foi removida.

Conceitualmente poderão existir informações como:

ReadAt
DismissedAt

A estrutura definitiva será definida na modelagem do domínio.

32. Remoção de Notificações

O usuário poderá remover notificações de sua própria lista a qualquer momento.

Também deverá existir uma ação para limpar o histórico pessoal de notificações.

A remoção deverá afetar somente o usuário destinatário.

33. Retenção das Notificações

Notificações poderão permanecer armazenadas por até 90 dias.

Após esse período, poderão ser removidas definitivamente por processamento automático.

34. SignalR

SignalR será utilizado para comunicação em tempo real.

Será responsável por entregar eventos a usuários atualmente conectados.

SignalR não será utilizado como mecanismo de persistência das notificações.

O sistema deverá garantir isolamento entre Tenants durante a comunicação em tempo real.

35. Dashboard

O dashboard deverá ser a principal tela após o login.

Deverá apresentar informações relevantes para o usuário, como:

- tarefas atribuídas;
- tarefas atrasadas;
- tarefas próximas do prazo;
- projetos em andamento;
- atividades recentes;
- comentários relevantes;
- notificações não lidas.

O objetivo será permitir que o usuário responda rapidamente:

"O que preciso fazer e o que aconteceu enquanto eu não estava acompanhando?"

36. Segurança

36.1 Senhas

Senhas de usuários nunca deverão ser armazenadas em texto puro ou em formato reversível.

Somente hashes seguros de senha deverão ser persistidos.

36.2 Credenciais

Credenciais de:

- banco de dados;
- serviços;
- tokens;
- chaves;

não poderão ser armazenadas diretamente no código-fonte ou em arquivos versionados contendo valores reais.

36.3 Isolamento Multi-Tenant

Usuários de um Tenant não poderão acessar recursos pertencentes a outro Tenant.

Essa validação deverá ocorrer no backend.

O frontend não deverá ser considerado mecanismo de segurança.

36.4 Autorização

Toda operação protegida deverá validar:

Identidade
+
Tenant
+
Perfil
+
Participação no projeto
+
Permissões específicas

quando aplicável.

36.5 Logs

Logs não deverão armazenar:

- senhas;
 - tokens completos;
 - segredos;
 - credenciais;
 - informações sensíveis desnecessárias.
-

37. Requisitos Não Funcionais

Segurança

O sistema deverá seguir boas práticas de autenticação, autorização, proteção de credenciais e isolamento de dados.

Testabilidade

Regras de negócio importantes deverão possuir testes automatizados.

Manutenibilidade

As responsabilidades do sistema deverão ser separadas em camadas e componentes apropriados.

Observabilidade

A aplicação deverá possuir:

- logs estruturados;
- identificação de requisições;
- health checks;
- métricas posteriormente.

Performance

Listagens grandes deverão utilizar paginação.

Consultas deverão ser analisadas quanto a desempenho.

Índices deverão ser criados conforme necessidade.

Escalabilidade

A API deverá evitar estado local desnecessário e ser preparada para execução em múltiplas instâncias quando aplicável.

38. Tecnologias Planejadas

Backend:

```
C#  
.NET  
ASP.NET Core Web API  
Entity Framework Core  
Npgsql  
PostgreSQL
```

Frontend:

```
React  
TypeScript
```

Qualidade:

xUnit
Testes unitários
Testes de integração

Infraestrutura futura:

Docker
Docker Compose
GitHub Actions
CI/CD

Tempo real:

SignalR

Possíveis tecnologias futuras, apenas quando justificadas:

Redis
MongoDB

39. Arquitetura Planejada

A solução deverá seguir inicialmente a seguinte separação:

```
Workflow
|
├── Workflow.API
├── Workflow.Application
├── Workflow.Domain
├── Workflow.Infrastructure
|
├── Workflow.UnitTests
├── Workflow.IntegrationTests
|
├── frontend
└── docs
```

Responsabilidades:

Domain
→ regras centrais do domínio

Application

→ casos de uso

Infrastructure

→ banco e serviços externos

API

→ comunicação HTTP e entrada da aplicação

40. MVP Inicial

O primeiro produto funcional deverá contemplar:

- multiempresa;
- usuários;
- autenticação;
- autorização;
- projetos;
- membros de projeto;
- permissões básicas;
- tarefas;
- responsáveis;
- colaboradores;
- tarefas disponíveis para assumir;
- fluxo de status;
- validação;
- comentários;
- histórico;
- Kanban;
- dashboard;
- chat geral;
- notificações internas;
- SignalR.

41. Funcionalidades Futuras

Inicialmente ficarão fora do MVP:

- subtarefas;
- dependências estruturadas entre tarefas;
- anexos;
- chats privados;
- chat por projeto;
- grupos de conversa;
- preferências avançadas de notificações;

- notificações nativas do Windows;
 - cliente desktop;
 - Redis;
 - MongoDB;
 - integrações externas;
 - aplicativo mobile.
-

42. Princípios do Projeto

O WorkFlow deverá seguir os seguintes princípios durante o desenvolvimento:

1. Não adicionar tecnologias sem justificativa.
2. Não armazenar segredos no código.
3. Não confiar no frontend para segurança.
4. Manter isolamento entre empresas.
5. Separar regras de negócio da infraestrutura.
6. Escrever código testável.
7. Registrar decisões arquiteturais relevantes.
8. Evitar exclusão física quando houver necessidade de histórico.
9. Priorizar rastreabilidade.
10. Construir primeiro um sistema simples e correto antes de adicionar complexidade.